Lecture 13: Asynchronous Client-Server Applications

Objectives:

- Learn about the **asynchronous** methods of the **Socket** & **Stream** classes
- Learn how to use the **asynchronous** methods to write **Asynchronous** client-server applications.
- User-Defined Asynchronous Methods

1. Asynchronous methods of the Socket & Stream class

So far, our use of the **Socket** class has been limited to the blocking methods, namely, ***Accept***, ***Connect***, ***Send*** and ***Receive***.

Similarly, our use of the **FileStream** and **NetworkStream** classes (both concrete subclasses of the abstract **Stream** class)**,** have been limited to the blocking methods, ***Read*** and ***Write***.

Each of these methods blocks the execution of a program until its operation is completed.

Blocking may be acceptable in a console application, but it is clearly undesirable in windows applications where controls are expected to be responsive while network communications are taking place.

So far we have solved the problem of blocking by using threads.

An alternative way of avoiding blocking, which is more efficient than using threads, is by using the asynchronous methods of the **Socket** and **Stream** classes.

Asynchronous methods split common tasks into two parts, one part is performed using **Begin** methods and the other by **End** methods.

The **Begin** methods start a task asynchronously – That is without blocking.

They take an instance of **AsyncCallback** delegate and an **Object** (used to pass state information) as an arguments. This is in addition to other arguments required by their synchronous peers

When the task is finished, they invoke the delegate passed to them

as argument.

The delegate in turn calls its registered method, which will then call the corresponding **End** method to complete the task.

The following are the asynchronous methods of the **Socket** and **Stream** classes tabulated based on the tasks they perform.

Socket Class:

Tasks Started by	**Task**	**Task ended by**
BeginAccept(…)	To accept an incoming connection	EndAccept(…)
BeginConnect(…)	To connect to a remote host	EndConnect(…)
BeginReceive(…)	To receive data from a socket	EndReceive(…)
BeginReceiveFrom(…)	To receive data from a host (UDP)	EndReceiveFrom(…)
BeginSend(…)	To send data to a socket	EndSend(…)
BeginSendTo(…)	To send data to a host (UDP)	EndSendTo(…)

Stream Class (Note: These methods are inherited by FileStream and NetworkStream classes)

Tasks Started by	**Task**	**Task ended by**
BeginRead(…)	To start reading from a stream	EndRead(…)
BeginWrite(…)	To start writing to a file	EndWrite(…)

To see how these methods are used, let as take the **BeginAccept** and **EndAccept** methods as example. The other methods work in a similar manner.

The signatures of the **BeginAccep** and **EndAccept** methods are as follows:

```
IAsyncResult BeginAccept(AsyncCallback callBack, Object
state)
Socket EndAccept(IAsyncResult result)
```

The *BeginAccept* method takes an instance of **AsyncCallback** delegate and an **Object** as parameters.

The Object instance is used to pass the state information to the ***Begin*** method and the ***End*** method.

The *BeginAccept* method, like all the other Begin methods, is non-blocking. That is, the statements following the methods will execute while the method performs its task in the background.

The *BeginAccept* method is said to complete its own part of the task when a client's request for connection is received. At this point, the **AsyncCallback** delegate instance is invoked.

The AsyncCallback delegate expects as argument, a void method with one parameter of type **IAsyncResult**.

When the AsyncCallback delegate is invoked, it automatically calls this method, passing it the result of the ***BeginAccept*** method through the **IAsyncResult** parameter.

The **IAsyncResult** instance received by the method associated with the AsyncCallback delegate has a property, **AsyncState** which represents the state information passed to the BeginAccept method, using the **Object** parameter.

The Socket instance is normally passed between the Begin and End methods using the state object parameter.

The following fragment shows how the BeginAccept and the EndAccept methods are used:

```
Socket server = new Socket(AddressFamily.InterNetwok,

SocketType.Stream, ProtocolType.Tcp);
IPEndPoint localEP = new IPEndPoint(IPAddress.Any, 9050);
server.Bind(localEP);
server.Listen(10);
server.BeginAccept(new AsyncCallback(CallAccept), server);
…
private static void CallAccept(IAsyncResult result) {
  Socket server =  (Socket) result.AsyncState;
  Socket client = server.EndAccept(result);
  …
}
```

Note that the **End** methods are usually called within the method associated with the **AsyncCallback** delegate so that they complete the asynchronous task.

The **IAsyncResult** received by the **End** methods is that returned by **BeginAccept** method.

The use of the other pairs of asynchronous methods is similar as the following example shows:

2. Creating an asynchronous TCP server

The following example uses asynchronous socket methods to implement a windows-based asynchronous TCP echo client-server application.

The Server

```
using System;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.Text;

namespace TcpApplications {
     public class AsyncEchoServer : System.Windows.Forms.Form   {
          private System.Windows.Forms.Button startServer;
          private System.Windows.Forms.TextBox txtPort;
          private System.Windows.Forms.Label label3;
          private System.Windows.Forms.TextBox statusBox;
          private System.Windows.Forms.Label label2;
          private System.Windows.Forms.ListBox resultBox;
          private System.Windows.Forms.Label label;

          private Socket server;
          private Socket client;
          private byte[] data = new byte[1024];

          public AsyncEchoServer() {
               InitializeComponent();
          }

          public static void Main()
          {
               Application.Run(new AsyncEchoServer());
          }

          void startServerClick(object sender, System.EventArgs e)
     {
               server = new Socket(AddressFamily.InterNetwork,
```

```
                                        SocketType.Stream, ProtocolType.Tcp);
                int port = int.Parse(txtPort.Text);
                IPEndPoint localEP = new IPEndPoint(IPAddress.Any, port);
                server.Bind(localEP);
                server.Listen(4);
                server.BeginAccept(new AsyncCallback(OnConnected), null);

                startServer.Enabled = false;
                statusBox.Text = "Waiting for Client...";
           }

           void OnConnected(IAsyncResult result) {
                try  {
                     client = server.EndAccept(result);
                     statusBox.Text = "Connected to:
"+client.RemoteEndPoint;
                     byte[] message = Encoding.ASCII.GetBytes("Welcome to
my Server");
                     client.BeginSend(message, 0, message.Length,
SocketFlags.None,
                                      new AsyncCallback(OnDataSent),
null);
                }
                catch(SocketException) {
                     CloseClient();
                }
           }

           void OnDataSent(IAsyncResult result)    {
                try {
                     int sent = client.EndSend(result);
                     client.BeginReceive(data, 0, data.Length,
SocketFlags.None,
                                         new
AsyncCallback(OnDataReceived), null);
                }
                catch(SocketException) {
                     CloseClient();
                }
           }

           public void CloseClient() {
                client.Close();
                statusBox.Text = "Waiting for Clients...";
                server.BeginAccept(new AsyncCallback(OnConnected), null);

           }

           void OnDataReceived(IAsyncResult result){
                try  {
                     int receive = client.EndReceive(result);
                     if (receive == 0) {
                          CloseClient();
```

```
                    return;
                }
                else {
                    string message =
Encoding.ASCII.GetString(data,0, receive);
                    resultBox.Items.Add(message);
                    byte[] echoMessage =
Encoding.ASCII.GetBytes(message);
                    client.BeginSend(echoMessage, 0,
echoMessage.Length,
                     SocketFlags.None, new AsyncCallback(OnDataSent), null);
                    }
                }
                catch(SocketException)
                {
                    CloseClient();
                }
            }

            void InitializeComponent() {
                //deleted
            }
        }
}
```

The Client

```
using System;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.Text;

namespace TcpApplications {
    public class AsyncEchoClient : System.Windows.Forms.Form   {
        private System.Windows.Forms.Label label3;
        private System.Windows.Forms.ListBox resultBox;
        private System.Windows.Forms.Label label2;
        private System.Windows.Forms.Button connect;
        private System.Windows.Forms.Label label4;
        private System.Windows.Forms.Button send;
        private System.Windows.Forms.TextBox newText;
        private System.Windows.Forms.Label label;
        private System.Windows.Forms.TextBox txtIP;
        private System.Windows.Forms.Button disconnect;
        private System.Windows.Forms.TextBox txtPort;
        private System.Windows.Forms.TextBox connectionStatus;

        private byte[] data = new byte[1024];
        private Socket client;

        public AsyncEchoClient() {
            InitializeComponent();
        }
```

```
        public static void Main()          {
             Application.Run(new AsyncEchoClient());
        }

        void connectClick(object sender, System.EventArgs e)   {
             try {
                  connect.Enabled = false;
                  disconnect.Enabled = true;
                  send.Enabled = true;
                  connectionStatus.Text = "Connecting...";
                  client = new Socket(AddressFamily.InterNetwork,
                                                 SocketType.Stream,
ProtocolType.Tcp);
                  int port = int.Parse(txtPort.Text);
                  IPEndPoint remoteEP = new
IPEndPoint(IPAddress.Parse(txtIP.Text), port);
                  client.BeginConnect(remoteEP, new
AsyncCallback(OnConnected), null);
             }
             catch (SocketException) {
                  CloseConnection();
             }
        }

        void OnConnected(IAsyncResult result) {
             try {
                  client.EndConnect(result);
                  connectionStatus.Text= "Connected to:
"+client.RemoteEndPoint;
                  client.BeginReceive(data, 0,
data.Length,SocketFlags.None,
                                             new
AsyncCallback(OnDataReceived), null);
             }
             catch (SocketException) {
                  CloseConnection();
             }
        }

        void OnDataReceived(IAsyncResult result){
             try {
                  int receive = client.EndReceive(result);
                  string message = Encoding.ASCII.GetString(data, 0,
receive);
                  resultBox.Items.Add(message);
             }
             catch (Exception) {
                  CloseConnection();
             }
        }

        void disconnectClick(object sender, System.EventArgs e) {
```

```
                CloseConnection();
            }

            void sendClick(object sender, System.EventArgs e){
                try  {
                    byte[] message =
Encoding.ASCII.GetBytes(newText.Text);
                    newText.Clear();
                    client.BeginSend(message, 0, message.Length,
SocketFlags.None,
                                     new AsyncCallback(OnDataSent),
null);
                }
                catch (SocketException) {
                    CloseConnection();
                }
            }

            void OnDataSent(IAsyncResult result)    {
                try  {
                    int sent = client.EndSend(result);
                    client.BeginReceive(data, 0, data.Length,
SocketFlags.None,
                                        new
AsyncCallback(OnDataReceived), null);
                }
                catch (SocketException){
                    CloseConnection();
                }
            }

            public void CloseConnection() {
                client.Close();
                connectionStatus.Text = "Disconnected";
                connect.Enabled = true;
                disconnect.Enabled = false;
                send.Enabled = false;
            }


            void InitializeComponent() {
                //deleted
            }
        }
}
```

3. User-Defined Asynchronous Methods

As we have seen from the above example, asynchronous methods can be very useful in writing responsive programs especially when

the operation being done will take long time or is beyond the control of the application.

A natural question is, can we write our own asynchronous methods? The answer is yes.

In .NET, you can call any method asynchronously. This asynchronous support is provided through delegates.

When you define a delegate (to associate the method), the common language runtime automatically defines **BeginInvoke** and **EndInvoke** methods that you can call in order to execute the referenced method asynchronously.

The **BeginInvoke** method initiates an asynchronous operation and returns immediately, returning a reference to an object that implements the **IAsyncResult** interface.

Your program can make use of the returned reference to monitor the progress of the asynchronous operation – using the **IsCompleted** property.

When the operation completes, your program calls the delegate's **EndInvoke** method to get the result.

The delegate's **BeginInvoke** and **EndInvoke** methods are created at the time of compilation and their method signatures depend on the delegate's method signatures.

For example, if you have a method called `LongOperation` that takes a single int parameter and returns an int, then you would define the delegate like this:

```
public delegate int LongOperationDelegate(int aParam);
```

The compiler automatically generates the following method signatures for **BeginInvoke** and **EndInvoke**:

```
public IAsyncResult BeginInvoke(int aParam, AsyncCallback
cb,
                                Object AsyncState);

public int EndInvoke(IAsyncResult ar);
```

BeginInvoke is generated to take two additional parameters beyond those used by your method: **AsyncCallback** and **Object**.

The parameters for **EndInvoke** are any *ref* or *out* parameters that your method takes, followed by an IAsyncResult parameter. It also returns the same type as your method.

The following code shows a complete example.

```
using System;

public class UserDefinedAsync
 {
    // define the delegate
    delegate int LongOperationDelegate(int time, ref int refParam);

    public static void Main()
    {
      // create an instance of the delegate
      LongOperationDelegate dlgt = new LongOperationDelegate(LongOperation);
      int refParam = 0;

      // call BeginInvoke to initiate the asynchronous operation
      IAsyncResult ar = dlgt.BeginInvoke(5000, ref refParam,
                                         new AsyncCallback(OnCompletion), dlgt);

      // do some work while waiting for completion
      while (!ar.IsCompleted)
      {
        Console.WriteLine("Waiting for completion");
        System.Threading.Thread.Sleep(1000);
      }

    }

     public static void OnCompletion(IAsyncResult ar) {
           int refParam = 0;
           LongOperationDelegate dlgt = (LongOperationDelegate)
ar.AsyncState;
      int longResult = dlgt.EndInvoke(ref refParam, ar);
      Console.WriteLine("longResult={0}", longResult);
      Console.WriteLine("refParam={0}", refParam);
      Console.WriteLine("Press Enter to exit the program.");
      Console.ReadLine();
     }


    public static int LongOperation(int time, ref int refParam)
    {
      // simulate a long-running operation
      while (time > 0)
      {
        Console.WriteLine("*LongOperation");
        System.Threading.Thread.Sleep(1000);
        time -= 1000;
      }
      refParam = 42;
      return 1;
    }
 }
```